

DOCUMENTO DE PROJETO

# Agent Orchestrator

Guia Técnico & Análise de Engenharia

Geração automatizada de ambientes Claude Code,  
otimizada por medição de custo e de contexto.

Guia completo do código · análise de engenharia · 2026

# Sumário

---

1. Para que serve
2. Glossário de termos
3. Como funciona — o fluxo completo
4. Arquitetura — as camadas
5. Passeio pelo código — módulo a módulo
  - 5.1 `utils/claude.py` — a ponte com o Claude
  - 5.2 `utils/analysis_cache.py` — cache da análise
  - 5.3 `utils/storage.py` — histórico de projetos
  - 5.4 `utils/agents_store.py` — biblioteca de agentes
  - 5.5 `utils/claude_sessions.py` — ler o consumo real
  - 5.6 `utils/verify.py` — o gate de testes
  - 5.7 `core/analyzer.py` — o cérebro da análise
  - 5.8 `core/builder.py` — monta os arquivos (sem tocar o disco)
  - 5.9 `core/writer.py` — escreve no disco
  - 5.10 `core/launcher.py` — abre o Claude Code
  - 5.11 `core/folderpicker.py` — o seletor de pasta
  - 5.12 `core/usage.py` — o medidor de tokens
  - 5.13 `core/dispatcher.py` — o executor do plano
  - 5.14 `api/` — a camada HTTP (fina)
  - 5.15 `frontend/` — as 3 telas
  - 5.16 `templates/` — biblioteca reutilizável
6. As decisões de design e a investigação de custo (o "porquê")
7. Como rodar e testar
8. Modelo mental para ensinar
9. O que ficou pendente (mapa do futuro)
  - Avaliação técnica do Agent Orchestrator: o que o torna sólido e a tese por trás dele
1. As forças do projeto
2. A descoberta central (a tese do projeto)
3. Veredito

# Super Guia do Agent Orchestrator

---

Guia completo para **entender e ensinar** este projeto: o que faz, os termos envolvidos, como o código funciona (módulo a módulo, função a função) e — o mais importante — **por que** cada decisão foi tomada.

Leia na ordem se está aprendendo; use como referência se já conhece. As seções 1–4 dão o modelo mental; a 5 é o passeio pelo código; a 6 é o "porquê" medido.

## 1. Para que serve

---

O Agent Orchestrator é uma **interface web** que **prepara um projeto para ser desenvolvido com o Claude Code**. Em vez de você abrir o Claude num diretório vazio e explicar tudo do zero, o orquestrador:

1. recebe uma **descrição em texto** do projeto;
2. analisa e gera automaticamente o **andaime de contexto**: um `CLAUDE.md` (manual do projeto), **subagentes** especializados, **hooks** de automação, um **primeiro prompt** e um **plano de tasks**;
3. deixa você **revisar e editar** tudo antes de criar;
4. **escreve os arquivos** no disco e **abre o Claude Code** já configurado — ou **executa o plano** task a task automaticamente (o "dispatcher").

A meta do projeto, em uma frase: **fazer o Claude Code produzir projetos melhores e gastando menos tokens** — automatizando o setup que normalmente é manual.

---

## 2. Glossário de termos

---

Entender estes termos é metade do projeto.

**Token** — a unidade que o modelo de linguagem lê e escreve ( $\approx \frac{3}{4}$  de uma palavra). Tudo que entra e sai do modelo é cobrado em tokens. **Economizar tokens** = **economizar custo/limite**.

**Contexto / janela de contexto** — tudo que o modelo "está lendo" naquele turno: o histórico da conversa, arquivos abertos, instruções. A cada turno, o modelo **relê todo o contexto acumulado**. Quanto maior o contexto, mais caro cada turno.

**Prompt** — o texto que você manda ao modelo. **Prompt encadeado**: vários prompts em sequência, onde a saída de um vira entrada do próximo (é o que o analyzer faz).

**Headless / `claude -p`** — rodar o Claude Code **sem interface interativa**: você manda um prompt, ele executa sozinho e devolve o resultado. O `-p` (de *print*) é a forma de chamar o Claude por

dentro de um programa.

**CLAUDE.md** — arquivo de "memória do projeto" que o Claude Code lê automaticamente ao abrir um diretório. Contém convenções, estrutura e regras. É **relido a cada turno**, então o tamanho dele tem custo recorrente.

**Subagente (ferramenta Task/Agent)** — o Claude pode abrir um *sub-Claude* para uma tarefa específica. O subagente roda em **contexto próprio e separado** e devolve só um resumo à conversa principal. **Delegação** = o agente principal mandar trabalho para um subagente.

**Isolamento de contexto** — a ideia-chave de economia: se o trabalho pesado roda num subagente (contexto separado e descartável), a conversa principal **não acumula** esse peso, e os turnos seguintes ficam baratos.

**Hook** — um script que o Claude Code dispara **automaticamente** num evento, sem o modelo decidir. Eventos: `PreToolUse` (antes de usar uma ferramenta — pode bloquear), `PostToolUse` (depois), `Stop` (ao fim do turno). **Matcher** = filtro de qual ferramenta dispara o hook (ex.: `Bash`, `Write`).

**Frontmatter YAML** — o bloco `--- name: ... description: ... ---` no topo de um arquivo de agente. É a `description` que faz o Claude Code **saber quando acionar** aquele subagente. Sem frontmatter, o agente não é acionado.

**Tier de modelo** — qual modelo Claude usar: **Opus** (mais forte e caro), **Sonnet** (equilibrado, ~5× mais barato que Opus), **Haiku** (rápido e barato). Escolher o modelo certo por tarefa é um grande lever de custo.

**Prompt caching** — o provedor guarda em cache partes repetidas do contexto. Por isso o uso de tokens se divide em: - **input** (fresco): tokens novos, preço cheio; - **cache\_creation**: escrita no cache (~1,25× input); - **cache\_read**: leitura do cache (~0,1× input — 10× mais barato); - **output**: o que o modelo gera (mais caro). Em sessões longas, o `cache_read` domina (releitura do contexto acumulado).

**DAG** — *Directed Acyclic Graph*. Aqui: o **plano de tasks** ordenado por dependências (ex.: "modelo de dados" antes de "serviços que usam o modelo").

**Contrato** — a interface/saída que uma task expõe para as tasks que dependem dela (ex.: "função `valida_cpf(cpf) -> bool` em `cpf.py`"). É o que permite construir cada task isolada e ainda encaixar tudo.

**Gate** — um "portão" de decisão ou verificação. Dois gates no projeto: o **gate de complexidade** (vale orquestrar este projeto?) e o **gate de testes** (a task passou nos testes?).

**Dispatcher** — o componente que **executa o plano** task a task, cada uma numa sessão isolada e no modelo do seu tier.

**WSL** — *Windows Subsystem for Linux*. O Claude Code roda no Linux (WSL); o orquestrador roda no Python do Windows e chama o Claude via `wsl.exe`.

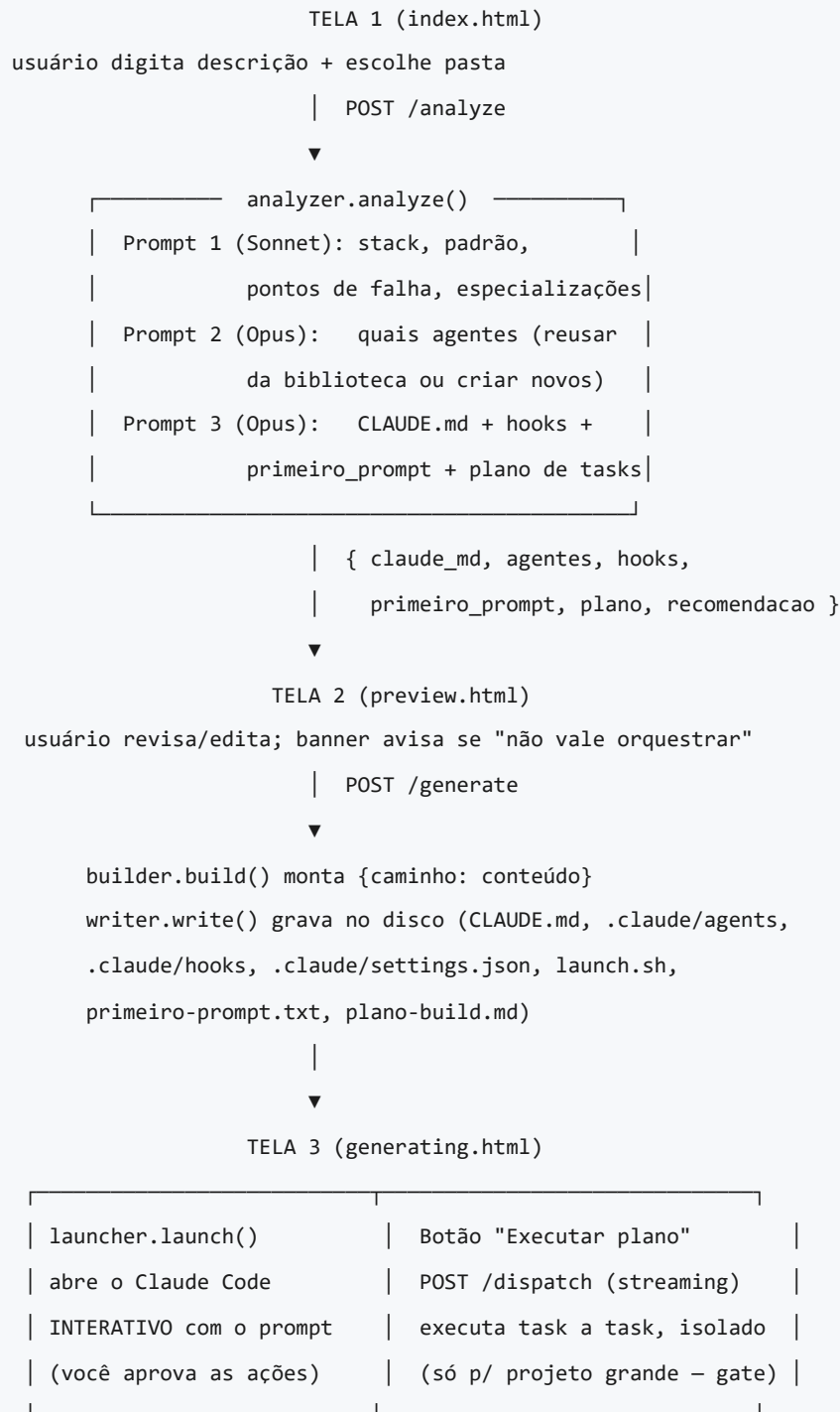
**FastAPI / endpoint / router** — o framework web. Um **endpoint** é uma rota HTTP (ex.: `POST /analyze`); um **router** agrupa endpoints.

**Pydantic / schema** — biblioteca que **valida dados** contra um formato esperado. Se o Claude devolver JSON faltando um campo, o schema rejeita com erro claro.

**NDJSON / streaming** — *Newline-Delimited JSON*: uma linha JSON por evento, enviadas **aos poucos** (streaming) em vez de tudo no fim. É como o dispatcher reporta progresso ao vivo.

---

### 3. Como funciona — o fluxo completo



Há **dois caminhos de execução** depois de gerar: - **Launcher**: abre o Claude interativo com o primeiro prompt (você acompanha). - **Dispatcher**: executa o plano automaticamente, task a task (recomendado só para projeto grande — veja a seção 6).

## 4. Arquitetura — as camadas

```
agent_orchestrator/
├─ app.py           # entrada: sobe o FastAPI, registra rotas, serve o frontend
├─ api/             # CAMADA HTTP — só recebe request e devolve response
├─ core/            # CAMADA DE LÓGICA — onde mora a inteligência
├─ utils/           # FERRAMENTAS — claude, storage, cache, etc.
├─ templates/       # biblioteca de agentes e hooks reutilizáveis
├─ frontend/        # HTML/CSS/JS puro (sem framework)
└─ tests/           # unit (cada módulo) + integration (a API inteira)
```

A regra de ouro do projeto: `api/` não tem lógica — só faz parsing de request/response e chama `core/`. Toda regra de negócio vive em `core/`. Todo acesso ao Claude passa por `utils/claude.py`. Isso mantém tudo testável e organizado.

## 5. Passeio pelo código — módulo a módulo

### 5.1 `utils/claude.py` — a ponte com o Claude

O ponto único por onde o projeto fala com o Claude Code. Tudo é subprocess (`ws1.exe claude -p`), nunca a API direta (não precisa de API key).

- `ClaudeError` / `ClaudeNotFound` — exceções. A segunda (CLI ausente) não deve ser repetida; a primeira (falha transitória) sim.
- `_neutral_cwd()` — cria uma pasta vazia temporária e roda o `claude -p` a partir dela. *Por quê*: se rodasse na pasta do orquestrador, o Claude carregaria o `CLAUDE.md` do próprio orquestrador em toda chamada de análise (medimos ~+4,5k tokens/chamada de lixo). Pasta vazia = sem vazamento.
- `_extract_json(raw)` — extrai o JSON da saída do Claude mesmo com prosa em volta. Tenta: (1) `json.loads` direto; (2) bloco markdown ````json`; (3) `raw_decode` a partir do primeiro `{` / `[` (pega o primeiro JSON completo, ignora texto antes e depois). *Por quê*: LLMs às vezes respondem "Aqui está: {...} 😊".
- `_run_once(prompt, timeout, model)` — uma chamada `claude -p`: monta o comando (com `--model` se houver), roda, checka erro, extrai JSON, garante que é um objeto.
- `run_prompt(prompt, timeout, model, retries)` — embrulha o `_run_once` com `retry` (repete em falha transitória; CLI ausente não repete). É o que o analyzer usa para os 3 prompts.
- `run_task(prompt, model, cwd, timeout)` — o runner do dispatcher. Diferente do `run_prompt`: roda no diretório do projeto (`cwd`) para enxergar o `CLAUDE.md` e os arquivos das tasks anteriores; usa `--permission-mode acceptEdits` (escreve arquivos sozinho); `--output-format json` para devolver uso de tokens e custo. Cada task é uma sessão separada → contexto isolado.

## 5.2 `utils/analysis_cache.py` — cache da análise

- `cache_key(descricao, templates)` — gera uma chave SHA-256 determinística a partir da descrição + lista de templates.
- `cache_get(key)` / `cache_set(key, value)` — leem/gravam o resultado em `~/orchestrator/cache/`. *Por quê:* gerar a **mesma** descrição de novo custa **zero** (reusa o resultado em vez de chamar o Claude 3 vezes).

## 5.3 `utils/storage.py` — histórico de projetos

- `save_project(pasta, files, primeiro_prompt)` — anexa o projeto ao `~/orchestrator/projetos.json`, sob um `threading.Lock` (evita corrida entre requisições) e via **escrita atômica**.
- `_atomic_write(path, content)` — grava num arquivo temporário e faz `os.replace` (rename atômico). *Por quê:* se o processo morrer no meio, o arquivo antigo fica intacto — sem janela de corrupção.
- `list_projects()` / `_load()` — leem o histórico (tolerante a JSON corrompido).

## 5.4 `utils/agents_store.py` — biblioteca de agentes

- `list_agents()` — junta os agentes da biblioteca (`templates/agents/`) + os globais (criados em projetos anteriores) + marca os **fixados** (*pinned*).
- `save_agents_from_files(files)` — quando um projeto é gerado, salva os agentes novos na biblioteca global, para reuso futuro.
- `set_pinned(name, pinned)` — fixa um agente para entrar em todo projeto novo.

## 5.5 `utils/claude_sessions.py` — ler o consumo real

- `to_wsl_path(path)` — converte `C:\Users\x` → `/mnt/c/Users/x` (Windows → WSL).
- `encode_session_dirname(pasta_wsl)` — reproduz como o Claude Code nomeia a pasta de transcript: **todo caractere não-alfanumérico vira -** (regra verificada).
- `read_session_jsonl(pasta_wsl)` — lê o transcript `.jsonl` da sessão daquele projeto (no Windows, via `wsl.exe`). É a base do medidor de tokens.

## 5.6 `utils/verify.py` — o gate de testes

- `run_pytest(pasta)` — roda `pytest` no projeto (**best-effort**): passou → ok; sem testes / `pytest` ausente / erro de coleta → não bloqueia; só retorna `False` quando há testes e eles **falham**. Usado pelo dispatcher para verificar cada task.

## 5.7 `core/analyzer.py` — o cérebro da análise

O coração: roda os 3 **prompts encadeados** e valida cada resposta.

- `_PROMPT_1/2/3` — os textos dos prompts (análise → agentes → CLAUDE.md+hooks+primeiro\_prompt+plano). Cada um pede **JSON puro**.

- **Schemas Pydantic** ( `_Analise` , `_Agente` , `_AgentesResult` , `_Hook` , `_TaskPlan` , `_Resultado` ) — definem o formato esperado de cada resposta. `extra="ignore"` tolera campos a mais.
- `_validar(model, data, etapa)` — valida a resposta contra o schema; se falhar, vira `ClaudeError` com mensagem clara (em vez de quebrar lá na frente).
- `_agentes_resumidos(agentes)` — versão enxuta dos agentes (nome + origem + resumo) para o Prompt 3, **sem reenviar o markdown completo** (economia).
- `_recomendacao(analise)` — o **gate de complexidade**: heurística que decide `orquestrar: true/false` pelo número de áreas de especialização ( $\geq 2 \rightarrow$  vale).
- `analyze(descricao)` — orquestra tudo: confere o cache  $\rightarrow$  Prompt 1 (Sonnet)  $\rightarrow$  Prompt 2 (Opus)  $\rightarrow$  Prompt 3 (Opus)  $\rightarrow$  monta a saída  $\rightarrow$  grava no cache. *Por quê dos modelos*: a análise (P1) é classificação simples  $\rightarrow$  Sonnet (barato); criar agentes (P2) e o artefato final (P3) precisam de qualidade  $\rightarrow$  Opus (medimos que o Sonnet gera agentes sem frontmatter e em inglês, quebrando a delegação).

## 5.8 `core/builder.py` — monta os arquivos (sem tocar o disco)

Recebe as strings e devolve um dicionário `{caminho: conteúdo}`. **Não escreve nada** — isso é do writer. Separar facilita teste e preview.

- `build(claude_md, agentes, hooks, primeiro_prompt, plano)` — monta: `CLAUDE.md` , `.claude/agents/<nome>.md` , `.claude/hooks/<tipo>-N.sh` , `.claude/settings.json` (registra os hooks), `.claude/primeiro-prompt.txt` , `.claude/launch.sh` e `.claude/plano-build.md` .
- `_LAUNCH_SCRIPT` — o `launch.sh` : faz `cd` no projeto e roda `claude --model sonnet "$(cat .claude/primeiro-prompt.txt)"` . *Por quê*: o prompt vem de **arquivo** (não da linha de comando) porque o `;` no prompt quebrava o Windows Terminal; e `--model sonnet` corta  $\sim 5\times$  no build.
- `_plano_md(plano)` — formata o plano de tasks como markdown legível.
- `_ensure_agents_section(claude_md, agent_files)` — **garante** uma seção `## Agentes` no `CLAUDE.md` listando os subagentes. *Por quê*: sem essa seção o Claude raramente delega.
- `_normalize_name(name)` — torna o nome do agente seguro para virar caminho de arquivo ( `[a-zA-Z0-9_-]` ). *Por quê*: impede **path traversal** (ex.: um nome `../../etc/passwd` vira `etc-passwd` ).
- `_resolve_agent(agent, name)` — se o agente é "biblioteca", lê o template; se é "novo", usa o conteúdo gerado.

## 5.9 `core/writer.py` — escreve no disco

- `check_conflicts(files, pasta)` — retorna quais arquivos **já existem** na pasta. *Por quê*: não sobrescrever sem confirmar (vira o HTTP 409).
- `write(files, pasta)` — grava cada arquivo, criando as pastas. Usa `newline="\n"` para **não** gravar CRLF (que quebraria os scripts bash no WSL).

## 5.10 `core/launcher.py` — abre o Claude Code

- `launch(pasta_wsl)` — abre um terminal (Windows Terminal → WSL → `bash launch.sh`) que roda o Claude interativo. *Por quê WSL:* o Claude vive lá.

## 5.11 `core/folderpicker.py` — o seletor de pasta

- `_windows_python()` — localiza o Python do Windows (para abrir o diálogo tkinter), com `autodetecção` em vez de caminho cravado.
- `pick_folder()` — abre um diálogo nativo de "escolher pasta" via subprocess. *Por quê subprocess fora do claude.py:* aqui o subprocess chama Python/tkinter, não o Claude — exceção justificada à regra.

## 5.12 `core/usage.py` — o medidor de tokens

- `_PRECOS` — tabela de preço (USD por milhão de tokens) por família de modelo.
- `_tier(model)` — classifica o id do modelo em opus/sonnet/haiku.
- `_custo_usd(por_modelo)` — calcula o custo em USD somando por modelo.
- `aggregate(jsonl_text)` — lê o transcript e soma o uso: `input_fresco`, `cache_creation`, `cache_read`, `output`, total e **custo em USD**. É o que mostra "quanto custou de verdade" um build.

## 5.13 `core/dispatcher.py` — o executor do plano

A peça que **força** a divisão (em vez de torcer para o modelo se auto-dividir).

- `ordenar(plano)` — ordenação topológica do DAG: tasks sem dependência pendente primeiro; **quebra ciclo sem travar** (se houver dependência impossível, emite o resto na ordem dada).
- `prompt_da_task(task, feitas)` — monta o prompt **enxuto** de uma task: leia o CLAUDE.md, faça **SÓ** esta task (com o contrato), aplique o agente indicado, não refaça o resto. *Por quê enxuto:* para o contexto da task não inchar.
- `dispatch(plano, pasta, runner, gate)` — executa cada task em ordem topológica via `runner` (injetado → testável), com `gate` opcional após cada uma. Lógica **pura**: o runner de produção (`run_task`) e o gate (`run_pytest`) são injetados.

## 5.14 `api/` — a camada HTTP (fina)

- `analyze.py` — `POST /analyze` → chama `analyzer.analyze` numa thread; erro do Claude vira HTTP 502.
- `generate.py` — `POST /generate` → `builder.build` + `writer.write`; se há conflito e `sobrescrever=false`, retorna **409** com a lista; depois grava o histórico e chama o `launcher`.
- `dispatch.py` — `POST /dispatch` → **streaming NDJSON**: roda o plano task a task, emitindo uma linha por task (com `testes_ok` do gate) + uma de resumo.
- `usage.py` — `GET /usage?pasta=...` → lê o transcript e devolve o consumo + custo.

- `projects.py` / `agents.py` / `folderpicker.py` — histórico, biblioteca de agentes (listar/fixar) e o diálogo de pasta.

### 5.15 `frontend/` — as 3 telas

HTML/CSS/JS puro (sem framework, por decisão). `index.html` (descrever), `preview.html` (revisar/editar, com o banner de recomendação), `generating.html` (resultado + botão do dispatcher que lê o stream e mostra progresso ao vivo).

### 5.16 `templates/` — biblioteca reutilizável

- `agents/*.md` — agentes prontos (test-writer, security-reviewer, etc.) que o Prompt 2 pode reusar em vez de criar do zero.
- `hooks/*.sh` — hooks prontos: `pre-bash.sh` (bloqueia comandos perigosos), `post-write.sh` (roda ruff em `.py`), `stop.sh` (roda pytest no fim do turno).

---

## 6. As decisões de design e a investigação de custo (o "porquê")

---

Esta é a parte que torna o projeto **inteligente**, e veio de medições reais.

O custo é dominado por **releitura de contexto**. Medindo builds reais, ~94% do gasto é `cache_read` — o modelo relendo o contexto acumulado a cada turno. Numa sessão longa, esse acúmulo cresce quase quadraticamente. O inimigo não é "ler caro" — é **reler a mesma coisa repetidas vezes**. Não existe "comprimir/criptografar" para ler mais barato: o cache já é o desconto máximo (~0,1×); só dá para (a) carregar menos ou (b) usar modelo mais barato.

Os levers de economia, em ordem: 1. **Modelo barato no build (Sonnet)** — ~5×, sempre vale, sem mudança estrutural. 2. **Isolamento de contexto** — subagentes ou tasks isoladas, para a curva de `cache_read` não inchar. 3. **CLAUDE.md enxuto** — relido todo turno; tamanho tem custo composto. 4. **Cache por hash** — não regerar o que já foi gerado.

A grande lição (medida): **orquestração é investimento que só se paga em projeto grande**. Em projeto pequeno ela custa MAIS: - Calculadora (~200 LOC): orquestrador **3,7× pior** que prompt único. - mini-tarefas (dispatcher): **4× pior** (3 sessões frias pagam 3× o boot; sem acúmulo para economizar). - Chamados (~4.000 LOC, build de **15,2M tokens**): aí o monólito explode em `cache_read` e o isolamento venceria.

Por isso o **gate de complexidade** existe: só recomenda orquestrar/despachar quando o projeto é grande o bastante.

Um achado importante e honesto: pedir delegação no prompt **não basta**. Mesmo com "abra um subagente" explícito, o modelo delegou ~1 vez em 52 turnos. Por isso o **dispatcher** existe — ele **executa** a divisão (cada task numa sessão separada), em vez de torcer para o modelo se dividir sozinho.

---

## 7. Como rodar e testar

---

```
# Setup (Python do Windows)
pip install -e ".[dev]"

# Rodar o servidor
python app.py                # http://localhost:8000

# Testes
python -m pytest tests/ -q    # 132 testes
python -m ruff check .        # lint
python -m mypy core/ utils/  # tipos (strict; alguns dict "crus" pré-existent)
```

Detalhe do ambiente: o orquestrador roda no **Python do Windows**; o `claude` roda no **WSL** (usuário brendon). Por isso `utils/claude.py` chama `wsl.exe claude`.

---

## 8. Modelo mental para ensinar

---

Se for explicar para alguém, conte nesta ordem:

1. **O problema:** preparar um bom ambiente Claude Code (CLAUDE.md + agentes + hooks) é trabalhoso e manual. Este projeto automatiza isso a partir de uma descrição.
2. **O fluxo:** descrever → (3 prompts geram o andaime) → revisar → gerar arquivos → abrir/executar.
3. **A separação:** `api/` fina, `core/` com a lógica, `utils/` com as ferramentas, tudo testável porque o builder só monta e o writer só escreve.
4. **A grande sacada:** custo de token = releitura de contexto. Subagentes/tasks isoladas cortam essa releitura — mas só compensam em projeto grande, então existe um gate para decidir.
5. **A honestidade:** cada decisão foi **medida**, não suposta — inclusive as que mostraram que orquestrar nem sempre vale.

Uma frase para fechar: "O orquestrador não economiza por gerar; economiza por fazer o projeto rodar num regime de contexto isolado — e só quando isso compensa."

---

## 9. O que ficou pendente (mapa do futuro)

---

- **#12 — Tier free (OpenCode):** rotear tasks mecânicas para modelos gratuitos, reservando o Claude para o trabalho crítico.
- **Calibrar o gate por dados:** registrar decisão + custo real medido para o gate aprender o limiar com o uso, em vez de heurística fixa.
- **Abort no dispatcher:** poder parar um build que azedou (economiza tokens). ``

# Análise de Engenharia

---

## Avaliação técnica do Agent Orchestrator: o que o torna sólido e a tese por trás dele

Esta análise parte do código real e dos números medidos durante o desenvolvimento. O foco aqui é o que sustenta o projeto e a engenharia de fundo — as forças e a tese central.

### 1. As forças do projeto

---

**Engenharia guiada por medição.** O traço que mais distingue este projeto é não assumir — medir. A maioria dos sistemas de agentes parte da premissa de que "mais agentes e mais autonomia = melhor". Aqui, cada decisão de custo foi testada antes de virar regra, inclusive as que contrariaram a intuição inicial. Isso eleva o projeto de "ferramenta" para "investigação de engenharia".

**Separação `builder` monta / `writer` grava.** O builder produz um dicionário `{caminho: conteúdo}` sem tocar o disco; o writer só persiste. Essa fronteira deixa toda a montagem testável sem I/O e permite o preview editável. É a melhor decisão arquitetural do projeto e aparece refletida na suíte de testes.

**`_neutral_cwd` — contra o vazamento de contexto.** Rodar a análise a partir de uma pasta vazia impede que o Claude leia o `CLAUDE.md` do próprio orquestrador e se confunda. Parece detalhe, mas economiza ~4,5k tokens por chamada e elimina uma classe silenciosa de erro (o modelo seguindo regras que não são do projeto-alvo).

**Modelo por tier (Sonnet na triagem, Opus na geração).** Decisão correta e fundamentada: a triagem (classificação) é barata e o Sonnet dá conta; a geração de agentes exige o Opus porque o Sonnet falha no frontmatter YAML — e frontmatter quebrado inutiliza o mecanismo de delegação. Não é economia cega; é economia onde não custa qualidade.

**Dispatcher + ordenação topológica (DAG).** Planos longos saturam o contexto: no 5º passo o modelo esquece as restrições do 1º. Quebrar o plano em sessões frias, na ordem de dependências, faz cada parte rodar com foco total e histórico limpo. É o componente determinístico que tira a opacidade do comportamento do modelo.

### 2. A descoberta central (a tese do projeto)

---

Três fatos medidos sustentam todo o design:

1. **~94% do custo é releitura de contexto ( `cache_read` ).** O inimigo não é "ler caro" — é reler a mesma coisa a cada turno, numa janela que só cresce. Não existe truque de compressão: o cache já é o desconto máximo ( $\sim 0,1\times$  do input). Só dá para **carregar menos** (isolamento) ou **trocar a taxa** (modelo mais barato).
2. **A composição de custo importa.** Pelos números reais de um build grande: `cache_read` (14,3M)  $\gg$  `cache_creation` (658k)  $>$  `output` (251k)  $>$  `input_fresco` (50k — o menor). Quem quer cortar custo deve mirar a curva de releitura e o preço por token (output/cache), não o input fresco.
3. **Orquestrar é investimento que só se paga em projeto grande.** Medido: em projeto pequeno a orquestração custa  $3,7\times$ – $4\times$  mais (o boot de 3 sessões frias supera qualquer economia, porque não há contexto acumulado para isolar). Num projeto de  $\sim 4.000$  LOC e build de 15,2M tokens, o monólito explode em `cache_read` e o isolamento venceria.

E um achado desconfortável, mas honesto: **pedir delegação no prompt não funciona de forma confiável**. Mesmo instruindo "abra um subagente", o modelo delegou  $\sim 1$  vez em 52 turnos — ele tem viés de persistência (prefere insistir na mesma sessão a abrir mão do controle). Por isso o isolamento precisa ser **forçado** pelo dispatcher, não sugerido.

---

### 3. Veredito

---

No estado atual, o Agent Orchestrator é melhor descrito como uma **plataforma de experimentação para otimização de custo e contexto em desenvolvimento assistido por IA** do que como um simples gerador de ambientes. A maturidade não está no volume de features, e sim na disciplina de **medir cada hipótese** — inclusive aceitando quando a medição contraria a intenção (orquestrar nem sempre vale). É essa postura, mais do que qualquer componente isolado, que separa o projeto de uma ferramenta comum.